```
// index.php

session_start(['use_strict_mode' => true]);
print "sessionId: " . session_id() . "\n";

# Simulate cookie previously set by malicious injection attack...
$ curl -v --cookie "PHPSESSID=abcdef123456789abcdef123456789ab" \
  http://localhost:8080/index.php
...
> Cookie: PHPSESSID=abcdef123456789abcdef123456789ab
< Set-Cookie: PHPSESSID=b91b04c21ca8e270aaab9e9b1d0db7c8; path=/
...
sessionId: b91b04c21ca8e270aaab9e9b1d0db7c8
```

Given that PHP did not manage to find any existing session data, we lose nothing by discarding the supplied `PHPSESSID` value. `session_start()` would normally (with a default config of `session.use_strict_mode = 0`) use the value supplied in the cookie. By using strict session IDs, we negate any possible cookie-based session adoption, and our user data is safe. An additional tactic sometimes seen is to capture the IP address (usually at login) of the client when the session ID is generated, which is exposed via `$_SERVER['REMOTE_ADDR']` – if you start seeing requests with that session ID but from a different address, then it can be flagged as possibly malicious (you might at least require the user to authenticate again).

Tidying Up

Session data is not intended to live forever, or at least a session ID is not (I've discussed why it's a good idea to frequently generate new session IDs). To prevent your filesystem (or memcached/Redis/SQLite, or whatever storage mechanism is persisting your sessions) from filling up over time, there is a *garbage collection* mechanism built into PHP. This mechanism does have some idiosyncrasies, however.

When a session is started (by any user request), PHP performs a calculation based on the ini directives `session.gc_probability` and `session.gc_divisor`. Default values for these directives are `1` (probability) and `100` (divisor), which works out to a 1 in 100 chance for a script that calls `session_start()` to trigger automatic garbage collection.